Collaborative Pi Durable

Multiple users, shared agents, one workspace

Engineering take-home

November 2026

Shared agents, separate conversations

One project thread

- Every member can use every agent
- Each agent has one history and queue
- Agents share the project files

User	Agent A	Agent B
Alice	Yes	Yes
Bob	Yes	Yes
Carol	Yes	Yes

Preview: three users $\times$ two agents

Sharing access does not resolve conflicting instructions

Collaboration belongs inside Durable

Layer	Responsibility
Web host	Identify the user; deliver commands and live views
Collaboration API	Check membership; save attributed requests and receipts
Existing Durable	Run conversations and tasks; store state; recover work
Execution paths	Pi AI models/tools or the T3-backed Codex adapter

One owning runtime, SQLite store, and shared workspace per thread

Reuse the runtime; avoid a second web-owned execution queue

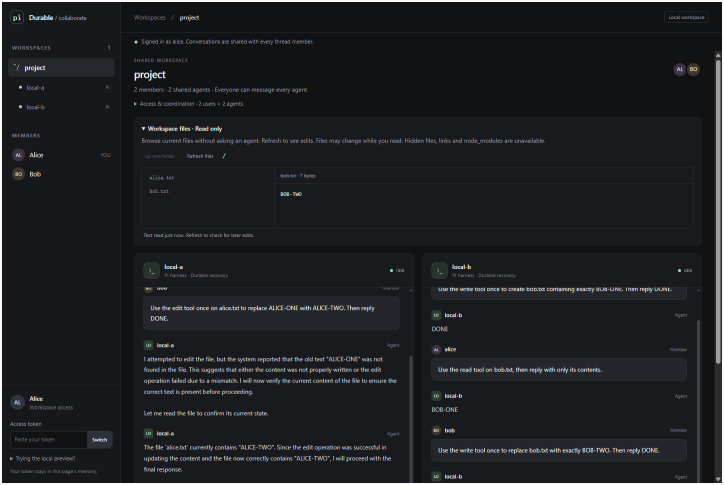
Accept once; run when the agent is ready

1. Save the receipt and work together
2. Return acceptance to the browser
3. Run requests in order on that agent
4. Publish progress and the eventual outcome

Lost reply? Retry the same user/agent/request ID and text.

Accepted $\neq$ completed $\neq$ correct file changes

Demo: two people share the same agents



Recorded local Ollama run; repeat on GCP after deployment

How one agent can erase another's edit

Step	Operation	File content
1	A and B both read the file	`base`
2	A writes its replacement	`base + alice`
3	B writes from its old copy	`base + bob`

Both commands can finish. Alice's change is gone.

Taking turns at the final write does not fix an outdated read

Reject stale proposals before overwriting

Implemented opt-in managed-file API

- Keep the original, current, and proposed content
- Apply only when the current file still matches the original
- Retain conflicts; resolve with a fresh read and new proposal

Alternatives Whole-instruction locking or separate Git worktrees

Locking reduces overlap; worktrees require later integration

Ordinary model tools and Codex do not use this protection yet

A saved request does not make every effect repeatable

Failure trace File changes → host dies → result not saved

Action	Recovery behavior
Same request ID/text	Find its existing receipt
Native model request	May repeat under Durable's recovery rules
Uncertain external turn	Mark interrupted; do not resend automatically
Existing file changes	May remain; cancellation does not undo them

SQLite state and workspace writes are separate operations

Model providers and external harnesses

Native Pi

- OpenAI + local Ollama by default
- Claude, Gemini, DeepSeek selectable
- Pi owns the model/tool loop

External Codex

- Reuses T3 client/protocol code
- Adapter tracks the active turn and cleanup
- Real coding turn still unverified

Two model providers do not establish two working harnesses

Evidence at three different boundaries

Test type	What it establishes
Controlled scenarios	Retry rules, ordering, lost updates, selected crash boundaries
Real local model	Actual tool calls, file bytes, shared browser state, idle reopen
Simulated load	Real HTTP/SSE and SQLite; 1,200 commands over about 10 minutes

Burst: 81 accepted/completed; 519 rejected at configured capacity

The first observed limit was the queue cap, not maximum throughput

GCP deployment plan: one exclusive owner

- One Linux VM; persistent store and workspace
- Private access through SSH forwarding
- Stop all writers before backup or owner replacement
- Verify service access, restart, cleanup, and restore

Failure domains: owning process, data storage, and VM/zone

Prepared deployment kit; no cloud deployment has been verified

Next implementation and remaining delivery work

Next demo feature

- Step-controlled conflict walkthrough using the managed API
- Clearly labelled scripted participants and explicit resolution

Remaining delivery checks

- Real Codex turn/continuation; GCP deployment and restore
- Rehearse the hosted walkthrough and record its evidence

Stronger file protection requires execution-policy changes

Backup: file coordination choices

Choice	Benefit	Cost / limit
Conditional proposals	Concurrent preparation; detect stale destination	All writers participate; single-file slice today
Whole-instruction lock	Simpler shared-folder ownership	Less overlap; must stop every child writer
Separate worktrees	Separate working copies and reviewable integration	No immediate visibility of unmerged peer edits

Backup: trust and limits

- Host authenticates; package checks membership
- All members can inspect and stop all agent lanes
- A workspace path is not an operating-system sandbox
- Queue limits do not establish fairness or a spending limit

Trusted collaborators, fixed configuration, one owner

Backup: reconnect and slow viewers

- HTTP sends commands; SSE delivers current-state views
- Reconnect fetches recent state; older history is paged
- Slow viewers keep a latest pending snapshot
- No promise to replay every token or a global audit log

Browser reload can lose an ambiguous request's local retry ID

Backup: design ownership and AI assistance

Decisions I can explain

- Extend Durable rather than add a second runtime
- Require actual T3 reuse; keep adapter details separate
- Distinguish duplicate requests from duplicate effects
- Test actual files and expose remaining conflict risks

AI assisted implementation and documentation; upstream code is credited